

Hotel Management System

Project Proposal

A cloud-based, multi-property reservation, billing and operations platform

Prepared for: [Client / Hotel Group Name]

Prepared by: Disura

Date: July 2026

Version: 1.0

Status: Draft for client review

Revision History

Version	Date	Author	Description
0.1	15 Jun 2026	Disura	Initial draft
1.0	01 Jul 2026	Disura	First version issued for client review

Table of Contents

1. Executive Summary

This document proposes the design, development, and deployment of a Hotel Management System (HMS) intended to replace manual or spreadsheet-based front-desk operations with a single, secure, cloud-based platform. The system will manage reservations, room inventory, guest profiles, billing, housekeeping, staff access, and reporting across one or more properties from a unified dashboard.

The proposed solution is built on a modern, cost-effective technology stack (React, Node.js, PostgreSQL) chosen specifically to balance total cost of ownership, long-term maintainability, and production-grade security, while remaining flexible enough to scale from a single boutique hotel to a multi-branch hotel group.

This proposal covers the business case, project scope, methodology, timeline, cost estimate, team structure, risk assessment, and success criteria for the engagement.

2. Background and Problem Statement

Many small and mid-sized hotels continue to rely on paper registers, spreadsheets, or disconnected point-of-sale tools to manage reservations, billing, and housekeeping. This creates several recurring problems:

- Double-bookings caused by manually cross-checking availability across channels.
- Delayed or inaccurate billing due to manually tallied guest folios.
- No real-time visibility into occupancy, revenue, or housekeeping status across properties.
- Limited or no audit trail for cancellations, refunds, and rate overrides.
- Inability to safely delegate access — staff often share a single login with full privileges.
- No structured way to expand to a second or third property without duplicating manual processes.

The proposed HMS directly addresses each of these gaps through a purpose-built, role-based, multi-property platform.

3. Project Objectives

1. Provide a single source of truth for room availability, eliminating double-bookings through database-enforced booking rules.
2. Automate guest billing (folio management) from check-in through checkout, including itemised charges and integrated payment collection.
3. Introduce role-based access control (Owner, Manager, Front Desk, Housekeeping, Accountant) so staff only see and do what their role permits.
4. Give management real-time occupancy, revenue, and performance reporting without manual data compilation.
5. Support multiple properties/branches from one account for hotel groups planning to expand.
6. Meet modern security and data-protection expectations, including encrypted credentials, audit logging, and secure payment handling.
7. Deliver a system that is cost-effective to run and straightforward for the client's own staff or a small IT team to maintain going forward.

4. Scope of Work

4.1 In Scope

- Web-based staff dashboard (reservations, guest profiles, billing, housekeeping, reporting, settings).
- Public-facing booking widget for direct guest reservations.
- Role-based authentication and authorization for staff accounts.
- Folio and billing module with integrated payment gateway (Stripe / PayHere).
- Housekeeping status tracking (clean / dirty / inspected / out-of-order).
- Multi-property support with property-level data isolation.
- Operational reports: occupancy, ADR, RevPAR, daily revenue, night-audit summary.
- Deployment to a managed cloud environment with backups and monitoring.
- User manuals and technical handover documentation.

4.2 Out of Scope (Phase 1)

- Direct two-way synchronisation with OTAs (Booking.com, Agoda, Expedia) — planned as a Phase 2 enhancement.
- Native mobile applications for guests (a mobile-responsive web app is included; native iOS/Android apps are a future phase).
- Point-of-sale for restaurant/spa outlets beyond basic folio charge posting.
- Accounting-system integration (e.g., QuickBooks/Xero export) — can be scoped separately.

5. Proposed Technology Stack (Summary)

A full technical breakdown is provided in the accompanying Software Design Document. In summary:

Layer	Technology	Rationale
Frontend	React 19 + TypeScript, Tailwind CSS	Fast, maintainable UI; large talent pool for future maintenance
Backend	Node.js + Express, TypeScript	Single language across stack lowers long-term maintenance cost
Database	PostgreSQL	Strong transactional integrity; prevents double-booking at the data layer
Cache / Queue	Redis	Session storage, rate limiting, background job queue
Hosting	DigitalOcean / AWS (managed)	Cost-effective, scales as the client grows
Payments	Stripe or PayHere	PCI compliance offloaded to a certified provider

6. Development Methodology

The project will follow an iterative, Agile-influenced approach delivered in short cycles (sprints), allowing the client to review working software early and often rather than waiting for a single final delivery.

8. Requirements gathering and sign-off (Software Requirements Specification).
9. System and database design (Software Design Document, Database Design Document, UI/UX Design Document).
10. Iterative development in 2-week sprints, each ending in a client demo.
11. Continuous testing (unit, integration, and user acceptance testing).
12. Staged deployment: development → staging → production.
13. Post-launch support and warranty period.

7. Project Timeline

Phase	Duration	Key Deliverables
Requirements & design	2 weeks	SRS, SDD, Database Design, UI/UX Design sign-off
Core development – reservations & rooms	3 weeks	Room inventory, availability engine, booking flow
Core development – billing & payments	2 weeks	Folio management, payment gateway integration
Core development – admin & reporting	2 weeks	RBAC, housekeeping module, reports
Testing & UAT	2 weeks	Bug fixes, client user-acceptance sign-off
Deployment & training	1 week	Production go-live, staff training, documentation handover
Warranty support	4 weeks post-launch	Bug fixes and minor adjustments at no extra cost

8. Cost Estimate

The figures below are indicative and will be finalised once the SRS is signed off. Hosting costs are recurring and separate from the one-time development cost.

Item	Estimate
Requirements, design & documentation	[Amount]
Core development (reservations, billing, RBAC, reporting)	[Amount]
Payment gateway integration	[Amount]
Testing, UAT support & deployment	[Amount]
Training & documentation handover	[Amount]

Item	Estimate
One-time development total	[Amount]
Recurring: cloud hosting & backups (monthly)	USD 60–300 depending on scale
Recurring: payment gateway fees	Per-transaction, set by provider (e.g., 2.9% + fixed fee)

9. Team and Responsibilities

Role	Responsibility
Developer (Disura)	Full-stack development, architecture, deployment, security hardening
Client stakeholder	Requirements sign-off, UAT participation, content and branding assets
Hotel operations lead	Validation of reservation, folio and housekeeping workflows against real operations

10. Risk Assessment

Risk	Likelihood	Impact	Mitigation
Requirements change mid-development	Medium	Medium	Formal sign-off of SRS before development; change requests tracked separately
Payment gateway approval delays	Medium	Medium	Apply for gateway merchant account in parallel with development
Data loss or corruption	Low	High	Automated daily backups, point-in-time recovery on managed database
Double-booking under concurrent load	Low	High	Database-level exclusion constraints, not just application checks
Staff resistance to new system	Medium	Medium	Hands-on training sessions and a simple, familiar UI

11. Success Criteria

- Zero double-bookings recorded during the first 90 days in production.
- Front-desk staff can complete a check-in and check-out without developer assistance after training.
- Daily/monthly occupancy and revenue reports are available without manual spreadsheet work.
- All payment transactions are reconciled automatically against folios.
- System uptime of 99.5% or higher outside of scheduled maintenance windows.

12. Conclusion

This proposal outlines a pragmatic, cost-effective path to replacing manual hotel operations with a secure, scalable management system. The accompanying Software Requirements Specification, Software

Design Document, Database Design Document, and UI/UX Design Document provide the detailed technical basis for this proposal and are intended to be reviewed and approved alongside it.

We welcome the opportunity to discuss this proposal further and adjust scope, timeline, or budget to best fit the client's priorities.